DeepL
订阅DeepL Pro以翻译大型文件。
欲了解更多信息，请访问www.DeepL.com/pro。

进行整合。开发重点是将这些组件用于机器人技术，并为机器人技术创建一个合适的对话管理系统（DM）。

为实现第 3.5 节所述的虚拟用户界面开发目标，我们采取了以下措施：

1. 功能适用性：为了开发多模态虚拟用户界面的目标，使搜救人员能够积极主动地与人交流，我们实施了必要的界面和功能。这一目标对软件设计的影响将在第 4.2 节中说明。

2. 可靠性：虚拟用户界面各组成部分的作用对系统的整体可靠性至关重要。正如第 2.2 章所述，虚拟用户界面处理链中任何一个环节出错都可能导致整个通信失败。一个明显的例子是，当 ASR 无法正确转录语句时，这就意味着后续的 NLP 组件无法正确捕捉语句的含义。

   因此，在设计和实施各个组件时，必须确保尽可能高的容错性。例如，ASR 组件应能抵御噪音，而 NLP 组件应能处理有转录错误的文本。

   除了在输入数据方面要有较高的容错率外，单个组件本身也必须具有尽可能低的错误率。要做到这一点，就必须全面分析各个组件和整个 VUI 的错误率。有关单个模块故障率详细信息，请参阅第 6、5 和 7 的相关内容。第 8 章对整个系统的可靠性进行全面评估。

3. 安全：安全措施以《个人数据保护法》（GDPR）的要求为基础。为此，所有数据均在机器人集成的计算机上进行本地处理。即使不存储过程数据（如语音记录、转录语句或触发操作），也能保证 VUI 的功能。有关数据保护合规性的更多详情将在另一节中讨论。

4. 可维护性：所开发的 VUI 使机器人开发人员实施需要语音通信的用例。每个模块都可以访问虚拟用户界面功能。有关模块系统及其结构的更多详细信息，请参见第 4.3.1 节。

5. 性能效率：选择处理所需的机器学习模型是为了让它们能在特区

的典型硬件上执行（见表 3.1）。最低要求是当前的 x86

要求处理器至少有四个内核（英特尔 i5 处理器，第 10 代或更新机型设置。

6. 用户友好性：所选择的 VUI 组件能够处理德语语言，并能日常用语和各种表述。确保 NutZQF 学习任何特定的语言命令，我们采用了当前基于深度学习的 NLP 方法。这些方法比基于字符串或关键词比较的方法更具优势。有关语言处理的更多详细信息，请参阅第 5 章。

7. 兼容性和可移植性：设计虚拟用户界面的目的是使其可移植到不同的应用程序中，并为集成到不同的系统中提供必要的接口。模块化结构和开放式接口使其成为可能。有关模块化结构和接口的详细信息，请参见第 4.2 和 4.3.J 章。

4.2 虚拟用户界面的组成部分

本章从技术角度概述了构成 VUI 的各个组件。此外，还解释了它们的一般功能。软件系统的分析分为三个阶段：系统背景、一般组件模型和详细组件模型。系统背景作为一个总体模型，代表了软件与外部系统接口的关系。这为理解软件与其他系统的交互以及描述软件与整个系统的集成提供了基础。另一方面，通用组件模型以更高的详细程度描述虚拟用户界面。它说明了构成 VUI 的组件，并清晰地展示了系统的整体结构。最后，详细的组件模型用于可视化单个组件的结构及其依赖关系。这样可以更深入地了解虚拟用户界面的功能和交互。本节旨在提供对虚拟用户界面组件及其功能的基本了解。在此阶段，这些组件被视为黑盒子。这些组件的具体功能和工作方法将在后续章节中详细介绍。

4.2.1 系统背景

图 4.1 VUI 及其与机器人硬件内外其他软件组件的接口。

在内部，语音接口与控制机器人执行器的软件包进行通信。例如，这包括用

于导航机器人或控制其头部和眼部运动的软件，这些软件在图中概括为接口设备。

虚拟用户界面还与机器人的传感器和分析传感器数据的软件组件相连接。其中一个例子是人员跟踪系统，该系统分析机器人摄像头的图像，并识别机器人环境中的新人员和已知人员。一旦识别到人，系统就会

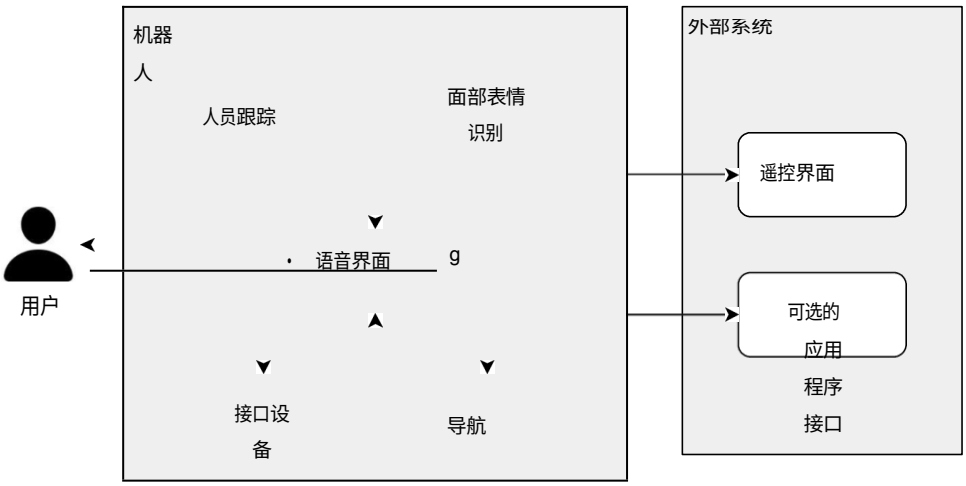


图 4.J：该模块视图显示了 VUI 及其在机器人硬件内外的接口示例。

人员跟踪系统会将这些信息转发给 VUI，然后 VUI 就可以向认识的人打招呼，或向不认识的人介绍自己。

VUI 的另一个重要功能是与机器人硬件之外外部系统进行通信。一方面，这涉及工作组开发的基于平板电脑的用户界面，该界面可实现对机器人的远程控制。用户通过平板电脑输入的控制指令必须由 VUI 执行，执行成功的指令必须反馈VUI。

还可以通过编程接口（API）与本地网络或互联网上的系统进行通信。与本地系统通信的用例包括向护理记录软件提供反馈，或与楼宇服务部门通信以控制照明或室内空调。与互联网的通信尤其适用于需要检索当前新闻或天气信息的情况。

与用户交流所需的音频数据评估以及通过扬声器输出语音反馈都直接通过 VUI 进行。通常情况下，所需的硬件连接到运行 VUI 计算机上。技术上可以通过网络将音频数据流从远程计算机的麦克风传输到 VUI。如果机器人本身没有足够的硬件资源来 VUI，这一点就很重要。在这种情况下，可VUI 的执行工作外包给网络中的计算机。此外，如果建筑物内安装了多个麦克风，而所有麦克风都要连接到集中式 VUI，也可以使用这种模式。这种分散操作模式需要一个

正常运行的网络，并增加了软件操作的复杂性。需要注意的是，网络连接必须可靠稳定，以音频数据的持续传输和 VUI 的高效运行。在同步多个麦克风和处理网络延迟时，可能会遇到更多挑战。

4.2.2通用组件模型

图 4.2 显示了虚拟用户界面的组件及其与外部系统的接口。如第 2.2 章所述，虚拟用户界面的结构基本上遵循文献中描述的标准模型。

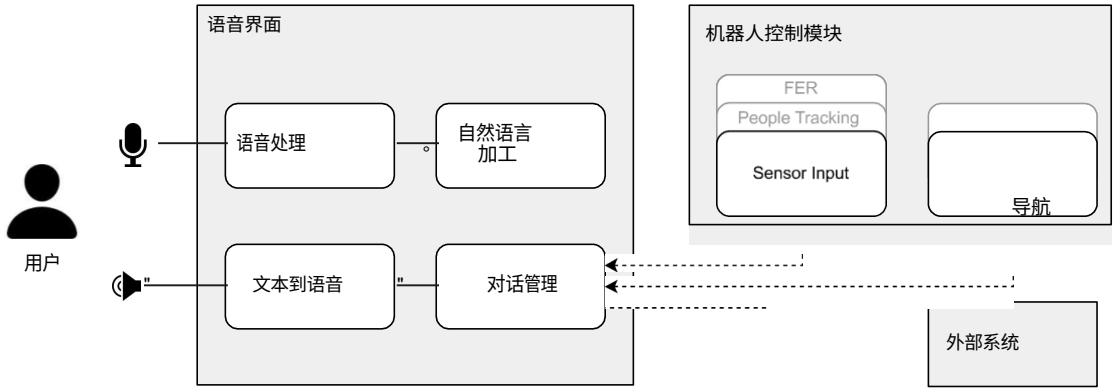


图 4.2：该积木式视图显示了虚拟用户界面的组件以及与外部系统的接口。下划线表示可选接口。

VUI 首先要进行语音处理，将音频数据流转换成文本。然后通过自然语言处理（NLP）对文本进行分析，以确定录制的语句与系统相关，以及是否可以将其分配给特定的操作。在文献中，这项任务也被称为意图分类。

对话管理（DM）负责执行语句定义的操作，并处理与外部系统的通信。它还可以执行多级操作。此外，软件模块可以分析机器人的传感器数据，并将其信息传送给 DM。然后，DM 决定是否以及如何对这些信息反应。

用户的回复文本也是在操作过程中形成的。因此，与 Heinroth 和 Minker 的模型[HM12, 第 12 页]（图 2.2），这里介绍的模型不包含单独的 "文本生成 "模块。

最后，TTS 模块制定的文本转换成音频信号通过扬声器播放。本地模型和在线服务都可用作 TTS。原型使用了微软的在线 TTS 服务，因为它的语音质量很高。

使 VUI 能够在没有网络连接的情况下工作，所有组件都必须具备离线功能。在开发 VUI 的过程中，使用了基于云的在线 TTS，因为在工作开始时还没有离线 TTS。

尽管使用的是基于云的 TTS，但仍可确保 VUI 能够没有互联网连接的情况下工作，具体做法是将系统可以提前做出的所有语句进行同步将其保存为音频文件。只要不在运行时动态创建新语句，就可以采用这种方法。如果需要在运行时生成语句，则需要一种即使没有网络连接也能在机器人硬件上本地合成语句的 TTS。

第 7.4 章介绍了离线 TTS 模型的技术。

4.2.3 详细的组件模型

图 4.3 显示了虚拟用户界面四个组件的结构，并解释了它们的内部结构。语音处理组件由语音活动检测（VAD）和自动语音识别（ASR）两个功能块组成。VAD 对音频数据流进行分析，以确定是否有人说话。如果识别到语音活动，就会记录语音，直到用户停止说话。然后，录制的语句被转发给 ASR，由 ASR 进行转录。ASR 转录录音是一个计算密集型过程。为了节省计算能力和能源，VAD 被连接到 ASR 的上游，这样 ASR 只有在实际说话时才会被执行。有关这一主题的更多详细信息，请参阅第 5 章。

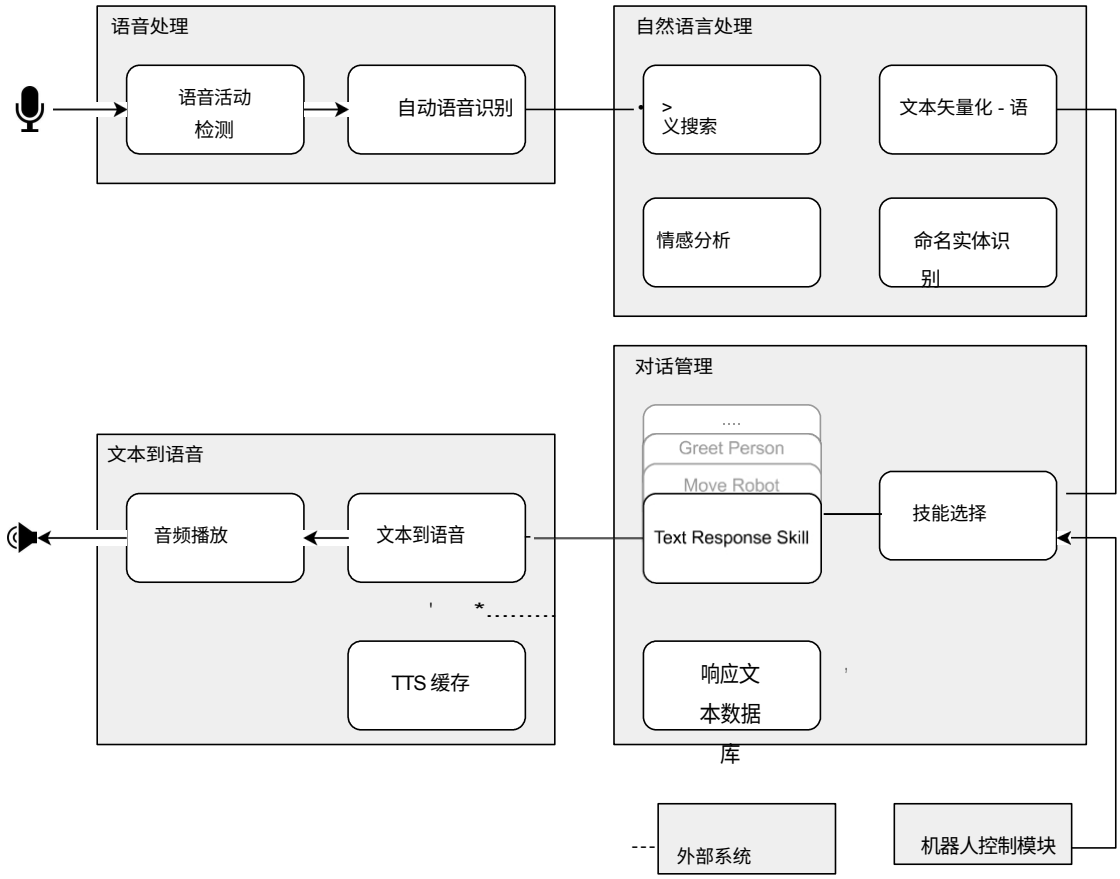


图 4.3：该组件模型显示了虚拟用户界面的组件结构以及与外部组件的接口。可选接口用断线表示。

自然语言处理（NLP）组件首先使用神经网络将识别到的用户语句转换成数字向量。然后该向量与一个已知用户语句数据库、一个

这一过程在文献中也被称为神经搜索。PFOZ6SSQS 的目的是在数据库中与语句相关联的行动（也称为意图）。然后将语句和意图转发给 DM。NLP 组件还包括用于分析文本情感（情感分析）和命名实体识别（NER）的模块，可识别文本文件中的人、地点或组织名称等特定信息。不过，这些功能并不适用于所有语句，只有在时才会执行。有关这方面的更多详情请参阅第 6 章。对话管理（DM）组件维护着一个称为技能的功能列表，这些技能可实现机器人在不同使用情况下的行为。根据为语句指定的意图，DM 会检查应执行哪种技能。此外，DM 还能对来自机器人软件组件的信号做出反应，例如，当电池电量不足或识别到特定人员时。在这种情况下，就会执行与事件相关的技能。系统发出的语句由相应的技能生成。该技能可以从数据库中选择语句或新语句，例如，它可以调用在线 API 获取天气数据，并根据收到的信息编写有关当前天气的语句。有关对话管理的更多信息，请参阅第 7 章。

最后，文本到语音（TTS）组件为用户生成音频输出。如果相同的语句已经合成，则从本地缓存中播放。如果没有，则使用 TTS 系统合成语句。然后通过扬声器。有关所使用的 TTS 模型的更多信息，请参阅第 7.4 章。

4.3 模块化、可扩展的互动

VUI 是一种计算机辅助系统（如社会辅助机器人 (SAR) 或数字助理 (DA)）能够对用户的陈述做出并自行陈述的技术。

所设计的虚拟用户界面提供了一个开发模型，可通过该模型实现各应用程序所需的功能。它为开发人员提供了可以实现新功能和新流程的接口。与智能手机的应用程序模式类似，开发人员可以通过定义的编程接口实现应用程序的特定功能。

4.3.1 通过技能进行互动

商业系统（如亚马逊的 Alexa bQCalc）将扩展功能称为 "技能"。Alexa 技能是对数字助理现有功能的扩展。例如，一个技能可以告诉用户天气预报。在这种模式下，技能还可以有参数，即所谓的 "槽"。插槽可以用户需要天气预报的地点。

这种开发模式有许多优点。它将 VUI 的功能与特定于应用程序的功能分开。技能和框架可以分开开发和测试。定义的接口结合了

这简化了技能的开发，因为其开发人员无需详细了解 VUI 的工作原理。

在本文中，技能一词的定义如下：

> *技能是语音用户界面功能的扩展是在标准化界面的基础上实现的。一项技能可为用户提供"超越语音用户界面标准功能"的特定功能或能力。*

为了使虚拟用户界面更好地适应各种应用场景，可以将多个单独的技能组合成一个使用场景组。这些组可以根据不同的应用场景激活，从而控制系统的行为，以适应相应的需求。单个技能可以是多个组的一部分。
目的是创建可用于不同应用的独立交互模块。图 4.4 举例说明了一些交互模式及其分组。对于护理院中的治疗支持或 AAL 功能等特定用例，可需要激活不同的组别。由于机器人不应被用于控制护理设施中的楼宇服务，因此在这种情况下，AAL 技能组可以停用。不过，这两种使用情况下的一般功能始终处于激活状态。

图 4.4：虚拟用户界面将独立的用例映射为技能。这些技能可根据需要调用，并可根据需要。一个用例所使用的不同技能可以组合在一起。根据应用场景激活这些组。

可以根据需要创建新的组别，系统也不局限于所介绍的三个组别。为了降低复杂性，暂时还不能同时激活不同的技能。控制技能的概念为机器人系统适应不同的使用情况和在不同的使用情况下利用单个功能提供了可能。它可以有针对性地激活某些组别，以控制系统的行为，并确保系统只执行与

相应用例相关的功能。在未来的工作中，这一概念可以扩展到同时激活不同技能的可能性，以进一步提高系统的灵活性。

早期的对话建模方法通常基于状态机或马尔科夫链，以确定哪些行动可以相互跟随 [Lux+13a]；

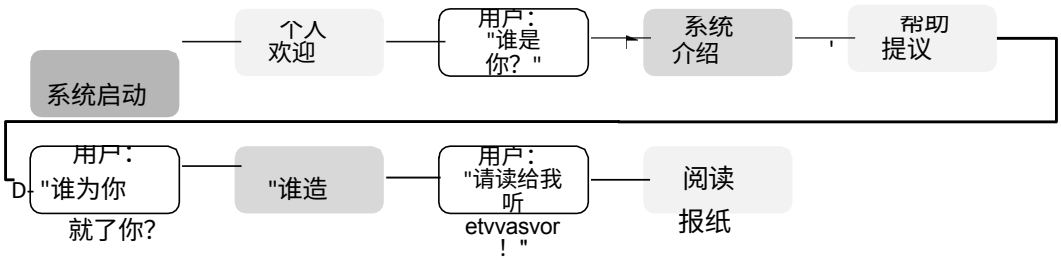


图 4.5：该图显示了一个人与虚拟用户界面之间对话流程的示例。人的发言显示在白色框中。对话是技能连锁执行结果，实现了连续的交互。

You06: Top+04] 使用这种限制是为了提高 ASR 的识别率。例如，可以规定用户用 "/a "或 "no "回答特定问题。

然而，工作小组的用户测试表明，在实践中，强调动作之间的转换，或者在本例中强调 SkillS, 6in [Pos+J 2]，是意义的。例如，人们在回答关于姓名的问题时会反问："你叫什么名字？根据这一经验，系统在设计时没有必要限制可能的下一步操作。相反，用户可以随时触发任何技能。

S€O.

图 4.5 显示了用户与机器人之间的对话示例。对话是使用各种技能的结果，每种技能都可以访问交互历史，并控制相应的操作，例如报告重复操作。下一章将介绍技能可用的接口，以便实现用例。

4.3.2 技能界面模型

虚拟用户界面为技能必要的组件和连接特区硬件的接口。技能实现的用例应尽可能映射为简短和自足的交互。这种模块化有两个主要目标：首先，在不同的应用场景中实现技能的可重用性，ZWQItQDS 将更复杂的交互划分为更小的单元，从而提高清晰度和可维护性。本节将详细介绍技能开发模型。它解释了如何使用这些技能来创建新的交互，以及有哪些接口可用于此目的。Python 编程语言用于实现 VUI 的技能。近年来，Python 已成为实施机器学习模型的标准。由于语言处理所需的软件包也是用 Python 开发的，因此

我们选择这种语言作为实现虚拟用户界面的基础。图 4.6 以 UML 组件图的形式概述了与技能相关的框架组件。

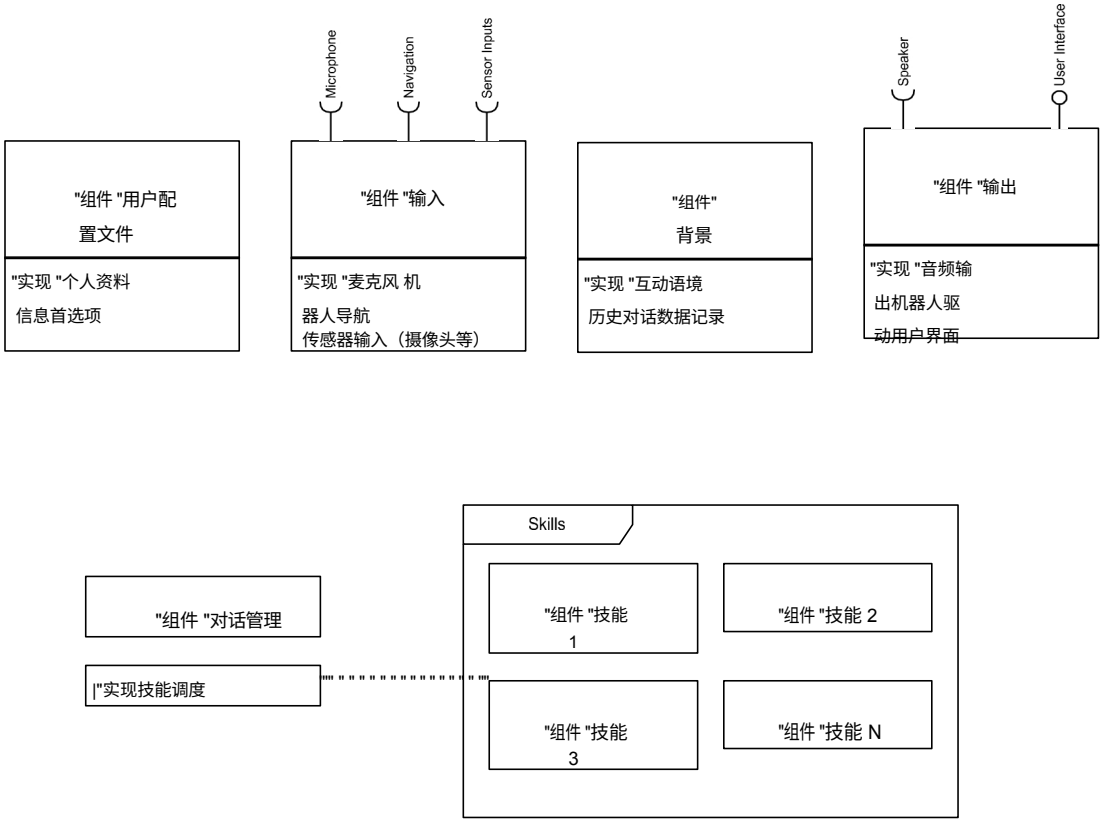


图 4.6：UML 组件图概述了框架中最重要的组件，用于扩展功能范围的技能模块概念。

对话管理 DM 模块负责执行技能。　任务包括：选择要执行的技能（调度），以及在有多个技能适用情况下确定优先级。技能激活由事件驱动。交互控制装置会处理所有接收到的事件，并选择适当的技能进行处理。事件可以由传感器直接触发，例如人接触到机器人的保险杠，也可以由传感器数据的机器学习模型间接触发，例如语音识别。其他例子包括通过用户的语音命令激活功能，或通过系统的摄像头识别人。

技能 每个技能都一种交互场景。例如，用户问候可以映射到一个技能中。然后，该技能包含的代码定义了对已知用户的问候语是其姓名，而对未知用户的问候语则是一般表述。技能的基本理念是人类与机器人之间的复杂交互分成尽可能小的单元。这样做的目的是降低复杂性。紧凑的交互场景更易于描述、开发和测试，这

类似于 Unix 的 "只做一件事并做好它 "的理念。如上一节所述，可以通过执行多个交互模式来创建更复杂的交互场景。通过上下文，技能可以选择识别当前和未来的交互模式。

历史传感器读数，并在执行时将其考虑在内。

上下文 上下文捆绑了所有用于检索数据和触发与用户交互的重要操作接口。要执行的操作存储在队列中，一旦资源可用，就会立即执行。上下文还包含一个用于记录机器人执行的操作的模块，并提供了一个访问这些数据的接口。之所以选择这种架构，是因为机器人中许多处理传感器信息的进程都是连续运行的。技能使用上下文来检索这些连续过程产生的最新结果。举例说明：在机器人运行期间，人员跟踪组件会持续识别机器人摄像头视野中人员。如果有新的人被识别出来或离开视野，输入模块就会收到通知，并将此信息存储在上下文中。如果某项技能需要了解机器人视野内有哪些人，它可以从上下文中获取这些信息

用户配置文件 用户配置文件模块管理用户有关的所有信息。它包含已知用户及其元数据的列表。元数据中存储了用户的特定信息，如所需的语音输出音量，以便系统能适应用户的需求。

输入和输出 这些模块为机器人的硬件和软件接口。通过它们可以访问传感器、执行器和机器人的基本功能。例如，这包括访问环境地图以及定位和路径规划。这些模块还访问外部机器学习（ML）模型。将对 gQnUtztQ ML 模型的访问捆绑在一个模块中，就可以在系统的其他组件中使用模型输出，而无需重新计算。此外，还可以关闭未使用的模型，以节省资源。

4.3.3 技能实施模式

如果在 VUI 中实施新的用例，必须创建两个人工制品：一个是要执行技能的语句定义，另一个是技能本身的实现。下面将通过一个示例来解释新技能的实现。在示例用例中，用户可以使用语音询问当前时间。

第一步是配置系统应响应的语句。源代码 4.J 中的几个示例就足以说明这一点。用户不必使用准确的措辞，因为系统不会对字符串进行比较。相反，

用户语句的语义将与示例语句的含义进行比较。这一程序还允许分配偏离的表述。有关该程序工作原理的更多信息，请参阅第 6 章。 对于评估中使用的技能（见第 8 章），每个技能都存储了 5 到 0 条语句。这意味着

在 93% 的情况下，用户都能触发所需的操作。在实际使用中，建议在初始列表系统无法正确识别的语句。

```
## tell time
  * 多晚了
3 * 什么时间
• 现在几点
• 请告诉我时间
6 – 现在几点
1
  请告诉我现在的时间
8 – 现在多晚了
9 – 德国现在几点了
0 – tell time
```

源代码 4.1：识别技能意图的配置。

技能（源代码 4.1）的语句定义采用 YAML 格式。一组语句可使用双散列（第一行）命名。这种命名是可选的，其作用是提供一个概览。第 1 0 行定义了分配给该组语句的语义标记。

当前时间的输出是通过源代码 4.2 所示的类实现的。 每个技能都是从 *BaseSkill* 派生的，并接收上下文作为参数。*技能*必须实现两个方法：*is suiteable* 决定技能是否可以接管事件的处理。在本例中，如果*事件*是语言语句，且已为其分配了 "telltime "意图，则函数返回 *True*。然后，DM 的调度程序会执行*运行*方法。技能可在此方法中执行任务。在本例中，系统时间被读出，并为用户制定了一个语句。该语句通过 *say context* 方法传送到 VUI。

```
1 类 TellTimeSkill（BaseSkill）
2     ：语境                          : Context)
3          ei.conte   "    ontex
5
6     async def is_suiteable(self)
7         事件        self.context.get_last event()
8
9           ==如果 isinstance(event,UtteranceDetectedEvent) 和 event.intent
              "telltime" 返回 True
1
12        返回 False
43
14
    async def run( self)
        =now dat et1me .
        =   时间 f" 现在是    (now . hour} clock (now . m1nut e} "
        self.context.say(time)
```

源代码 4.2：用户可要求显示当前时间的技能示例代码。

任何源代码都可以在*运行*方法中执行。如果要通过网络从外部系统获取数据，必须检查网络连接是否。这种检查应在 *is suiteable*

方法。如果该技能在没有活动网络连接的情况下无法工作，它可以在此时将 *False* 返回到 VUI，表示它用。

5 语音识别

5.1 从口语到文字

语音识别的任务是转录口语陈述。为此，必须首先通过麦克风录制音频信号。然后，软件可以进行语音识别，这在文献中也被称为自动语音识别（ASR）。目前的 ASR 方法基于深度神经网络，被称为 ASR 模型。这些模型将录音转换成文本。语音活动检测（VAD）可确定音频信号是否包含语音，只有当音频信号中确实包含语音时，才能执行 ASR 模型。只要没有人与语音用户界面（VUI）进行交互，这种方法节省处理能力和能源。该主题的详细介绍，请参阅以下章节
5.2 找到。
ASR 转录的准确性通常由单词错误率 WER) 决定。WER 的计算方法是将错误数（插入、删除和替换）除以参考文本中的字数。一般来说，模型大小与 WER 之间存在相关性。参数越多、规模越大的模型，WER 越低但推理时间越长。因此，在计算时间和准确性之间找到一个折中点非常重要。有关这一主题的更多信息，请参见第 5.3 节。

5.2 语音活动检测

语音活动检测（VAD）通过区分语音和非语音（如背景噪声或静音），在音频信号处理中发挥着至关重要的作用。这一功能对语音识别系统的效率和准确

性。VAD 系统能识别音频信号中真正包含语音的部分。这样，下游系统（如语音识别算法）就能将注意力集中在音频信号的相关部分。这项技术在以下情况下尤其有用

音频信号可用于长时间无声或有无关噪音的情况。

自动语音识别（ASR）模型通常是资源密集型的，而 VAD 可使这些模型得到更有效的利用。通过识别相关语音片段并将其转发给 ASR 系统，可避免因处理非语音片段造成的资源浪费。

为实现 VAD，人们开发并发布了各种方法，这些方法基本上可分为两类。一方面是分析音频信号某些特征的算法方法。另一方面是数据驱动方法，其中机器学习，特别是深度学习方法的使用多。

VAD 的统计程序通常从检测和过滤环境噪音开始[Coh03]。第一阶段结束后，分类器算法会根据选定的特征来决定手头的音频材料是否为语音[SKS99]。这种两阶段方法是语音识别领域传统统计方法的特点。相比之下，基于深度学习的方法只执行基本的预处理步骤，例如对音频信号进行归一化处理。然后，神经网络接管相关特征的选择和音频录音的分类工作 [HM3]。

在选择合适的 VAD 方法时，一个至关重要的方面是其处理流音频数据的能力。这包括将连续音频数据流直接输入模型并对其进行分析的能力。一些最精确的方法，[Bre23] 和 [Gao+23]，需要一个封闭的音频文件进行分析，因此不适合处理连续的音频数据。因此，这些模型不能用于 VUI。

WebRTC VAD

谷歌Chrome 浏览器开发WebRTC VAD 是 VAD 统计方法的现代实现。这种实现方式是 WebRTC 标准的一部分，该标准专为基于语音的实时通信而设计[1]。

WebRTC VAD 在机器人技术中的应用非常有趣，因为它提供了一种稳定的、与平台无关的解决方案。

独立实施。

在开发阶段，对模型识别语音的能力进行测试评估。结果发现，虽然模型能够识别语音，但其他声音也错误地归类为语音。图 5.J 中的三个图表说明了这一点，这三个图表模型在三种不同情况下的输出结果：语音、声学音乐和街道噪音（通过倾斜的窗户在室内录制）。在所有三种情况下，模型都错误地显示为

语音。

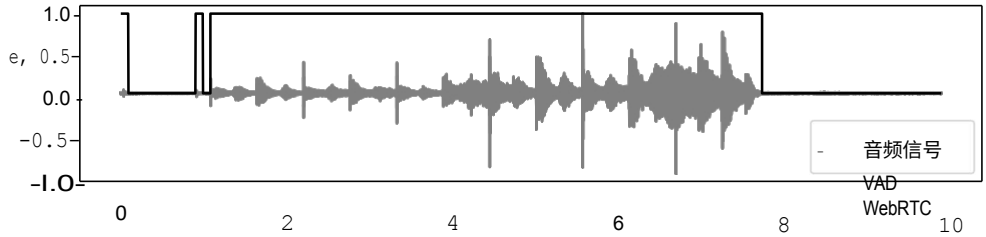
这种不正确的分类有两个主要后果。首先，au

错误地将不包含语音的录音转入语音识别 (ASR)。

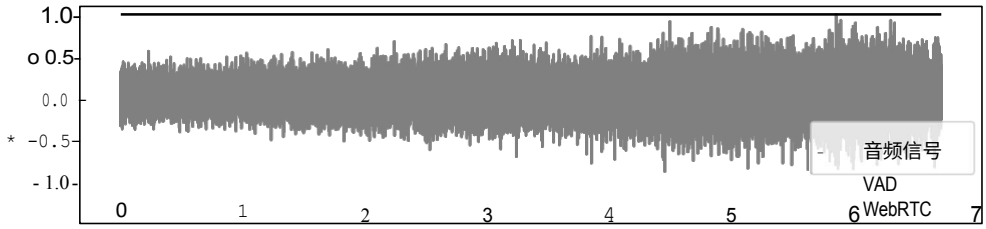
传输。这就导致 ASR 的执行频率过高，反过来又会最大限度地减少

'https://chromium.googlesource.com/external/webrtc/+/master/common audio/vad/

(a) 辐条



(b) 背景音乐响起



(c) 通过打开的窗户听到室内的街道噪音

图 5.J : 图形显示了 WebRTC 模型在三种不同场景下的输出结果：语音、音乐和街道噪音。模型输出值为 "语音"，0 表示 "无语音"。模型正确处理了语音录音。在没有语音的两种情况下，模型也显示语音已被识别。

硬件的处理能力。其次，在测试中，这导致 ASR 模型为实际上不包含任何语音的录音生成文本输出。这些被错误识别的文本导致了虚拟用户界面的错误激活。

为了解决这个问题，我们采用了语音活动检测技术。

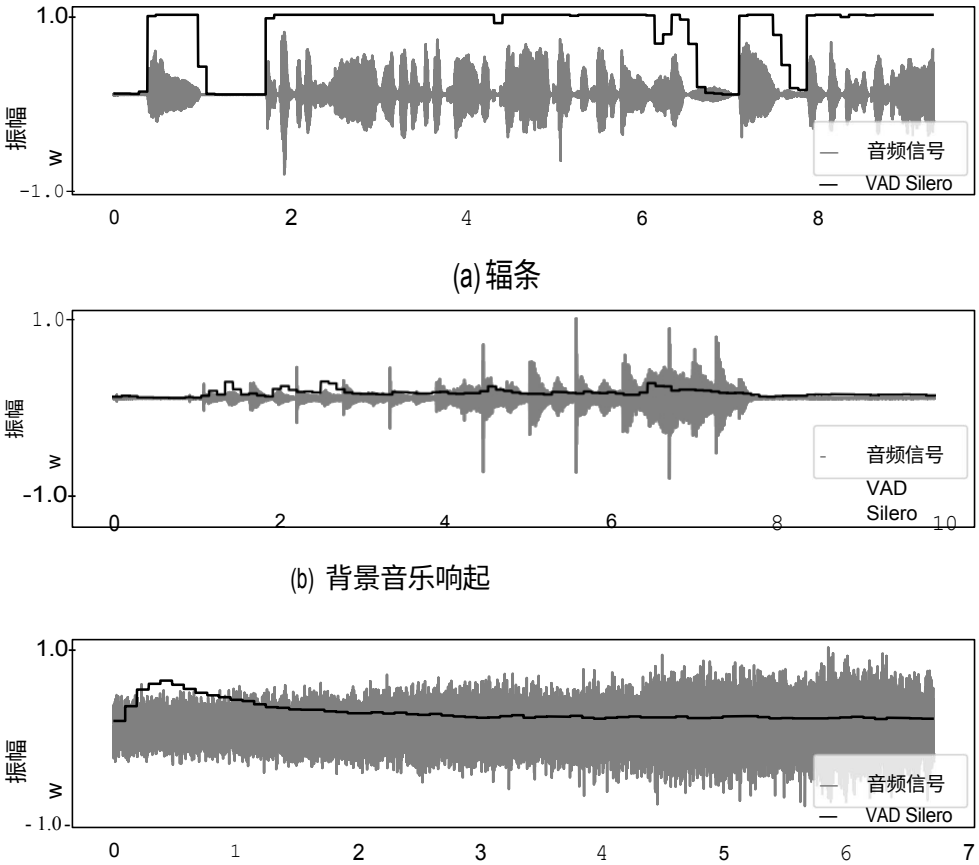
(VAD) 能有效区分语音和非语音。

Silero VAD

Silero VAD 是一个基于深度学习的 VAD 模型，在开源许可下公开。该模型的作者称，Silero VAD 的处理速度与 WebRTC VAD 。每 30 毫秒音频数据的处理速度不到一毫秒。

WebRTC VAD 只返回二进制值（有语音/无语音），相比之下，Silero VAD 返回的概率值介于 0 和之间，其中 1 表示有语音，0 表示无语音。图 5.2 显示了 Silero VAD 模型应用与 WebRTC VAD 相同音频数据的结果。可以看出，该模型不包含录音中的语音成分。

https://github.com/snakers4/silero-vad



(a) 辐条

(b) 背景音乐响起

(c) 透过倾斜的窗户听到室内的街道噪音

图 5.2：图形显示了 Silero 模型在三种不同情况下的输出结果：语音、音乐和街道噪音。该模型提供了 0 和 1 之间的概率，即这是语音。语音录音被正确识别，但语音中存在停顿的间隙需要进一步处理。在没有语音的两种情况下，模型给出的语音概率低于 60%。

正确识别。语音中的停顿有明显的间隙，可以通过额外的滤波器弥补。无论是音乐还是街道噪音，语音的概率都低于 60%。通过设置一个阈值，例如 90%，就可以确定音频是否包含语音！是否包含语音。

由于这些良好的质量结果，我们选择了 Silero VAD 来实施虚拟用户界面。

5.3 自动语音识别

在自动语音识别（ASR）的历史发展中，模型一般分为两类：大词汇量自动语音识别系统 (LVASR) 和小词汇量自动语音识别系统 (SVASR)。大词汇量系统是能够

识别大量词汇，这些词汇可能包括数万个单词。这些系统的缺点是需要大量计算资源才能有效。BiS ZUF 出版社的《大词汇系统》（BiS ZUF publication of mo

虽然有 zilla 的 "深度语音 "项目[3]和 Wave2Vec [Sch+19; Bae+20] 等模型，但德语仍没有强大的 LVASR 模型。

由于这些限制，移动机器人主要使用小词量自动识别（SVASR）模型。这些模型通常只能区分几百到几千个单词，但所需的计算资源较少。这种方法的缺点是 VUI 用户词汇量有限，因为他们只能利用这些有限的词汇量来进行查询。为了更方便地处理有限的词汇，有人提出了定义特定语音命令来触发操作等解决方案。然而，这种方法要求用户学习这些语音命令，这与要求 6.2 背道而驰，根据要求 6.2，系统应能够在不学习固定语音命令的情况下进行管理。

因此，这项工作使用 LVASR 模型，该模型应能自然地转录德语语句，并尽可能减少错误。自 2020 年以来，功能强大的德语模型的可用性有了显著提高，这为这一方法的实施提供了支持。

最近，人们提出了许多大型词汇自动识别（LVASR）架构，并提供相关的德国模型。其中Wav2Vec [Sch+19; Bae+20]、Conformer Transducer [Gul+20]、Citrinet [Maj+21]、Whisper [Rad+22]、Fast Con- former [Rek+23]、MMS [Pra+23]。如表 5.1 所示，其中一些架构有不同的尺寸。

表 5.1：表中列出了可使用德语模型的体系结构及其参数数量。单语言模型每个模型处理一种语言，多语言模型可以用一个模型处理所有语言。所有参数规格均以百万为单位。

结构	参数	低音小号 244M 低音中号 769M 低音大号 1550M
成型机	120M	
Citrinet	142M	
快速成型器 115M MMS		
	1	
000M		
Wav2Vec Basic	94.4M	
Wav2Vec Large	315M	
Wav2Vec IB	1	
000M		

语言	单语、7 种语言 + 社区模式 单语、7 种语言 + 社区模式 单语、7 种语言 + 社区模式 多语种、99 种语言
单语，9 种语言	多语种，99 种语言
单语，5 种语言	多语种，99 种语言
单语和多语种，10 种语言	
多语种，102 种语言	

自 2022 年以来，多语言 ASR 模型已成为一种趋势。这些模型通过一个训练有素的模型转录多种语言。不过，它们参数也比单语模型多，这通常会推理时间延长。

针对这项工作的具体目的，所选择的模型应满足两个基本标准：应达到尽可能高的精确度和

3https://github.com/mozilla/DeepSpeech

可以在给定的硬件上快速执行。因此，在选择合适的模型时需要慎重考虑，以便在速度和精度之间合理的折衷方案。现有的模型种类繁多，可以根据应用的要求确定模型。

5.3.1 计算

目前的 ASR 模型在 LibriSpeech [Pan+J 5] 等基准测试中的词错误率明显优于人类。同时，[GN2J] 和 gak22]等评估报告指出，语音理解错误是在实际应用 VUI 时遇到的最大问题。

诸如 [WP22] 等基准测试表明，ASR 模型在其所训练的数据集上取得了良好的结果。然而，当这些模型在其他数据集的音频数据上进行评估时，结果就会大打折扣。例如，德国的 Wav2vec 模型在其评估的 Common Voice 数据集上取得了 % 的 WER。而在包含政治家演讲VoxPopuli 数据集 [Wan+2] 上，WER 则翻了一番，达到 22%。

WP22] 的一项综合研究比较了当前的自动语音识别 (ASR) 模型。研究分析了德国现有模型的各种架构，包括 Citrinet、ContextNet、Conformed CTC、Conformed Transducer、Wav2Vec 2.0 和 Quartznet。总共 4 个不同的数据集用于评估。结果发现，在未经训练的数据集上进行测试时，模型的性能会明显下降。Radford 等人[Rad+22]也描述了这一现象。WP22] 发现的另一个问题是训练数据集的规范化不一致。例如，数据集中数字有时转录为数字词，有时转录为数字。没有一个模型能在所有测试场景中都表现出色。Conformed Transducer ZQ 的单词错误率（WER）最低，但在某些情况下会出现转录错误，如漏字或换字。相比之下，Wav2Vec

2.0 平均 WER 值较高，但仍更接近转录的原始内容。

自本研究报告发表以来又有了新的发展。内

特别是 Whisper 模型 [Rad+22] 的作者声称，他们通过在大型数据集上迁移学习，解决了域外数据性能下降的问题。

之所以需要进行单独的评估，是因为我们的目标是为 VUI 找出一个合适的模型。我们的目标是找到在训练过数据集和未知数据集上字错误率都很低的模型此外，这些模型必须能够在 SAR 标准硬件，特别是标准 CPU 上运行。

评估使用了两个数据集：通用语音数据集 [Ard+20] 和 MINDS14 集 [Ger+2]。通用语音数据集被域内数据集，因为所有评估模型都是在该数据集上训练的。

而 MINDS14 数据集则被用作域外数据集。相比之下，MINDS14 数据集被用作域外数据集。据了解，没有一个被评估的模型是在这个数据集上训练的。Mozilla Common Voice 是一个众包项目，志愿者可以通过该项目的网站 [4] 提交文本并审查其他人的贡献。第 13 版的数据集包含 1,257 个小时德语录音。MINDS-14 是一个数据集，包含对电子银行应用程序的口头请求。德语用户发言有 61 1 条。

为了全面了解模型的性能，对整个数据集的推理时间进行测量。由于某些架构存在多个模型，可能会产生不同的结果，因此在对这些架构进行评估时多个模型。

由于测试范围广泛评估了 15 个模型和 16 753 个音频文件，因此使用了 Nvidia RTX 4090 GPU 进行评估。

表 5.2：使用通用语音 13 数据集的可用 ASR 模型比较。推理时间指是完整数据集的推理时间。

型号名称	世卫组织	CER	推理时间（秒
NVIDIA/STT DE FASTCONFORMER_HYBRID 大型 PC	0.064	0.017	645
STT DE CONFIGER 大型转换器	0.068	0.020	855
oliverguhr/wav2vec2-large-xlsr-53-german-cv9	0.073	0.022	875
stt_en_contextnet_1 024	0.075	0.022	738
bofenghuang/whisper-medium-cv11-german	0.075	0.023	5901
openai/whisper-large-v2	0.080	0.029	5496
jonatasgrosman/wav2vec2-xls-r-1 b-german	0.084	0.025	1222
STT_EN_CITRINET_1 024	0.089	0.027	418
jonatasgrosman/wav2vec2-large-xlsr-53-german	0.090	0.026	878
oliverclock/wav2vec2-base-german-cv9	0.098	0.030	754
openai/whisper-medium	0.100	0.044	4190
bofenghuang/whisper-small-cv11-german	0.120	0.038	3257
aware-ai/whisper-small-german	0.132	0.037	2831
facebook/mms-1 b-all	0.135	0.035	1166
openai/whisper-small	0.158	0.056	2794

虽然 WER 通常被用作衡量语音识别模型性能的主要标准，但在某些应用中，转录语义的正确性比正字法的准确性更为重要。神经搜索中的一个例子说明这种偏好。使用单词向量通常容忍搜索查询中的微小输入错误。由于相似词语向量在向量空间表示法中彼此接近，因此转录中的微小偏差并不一定会导致错误的分类或不准确的搜索结果。因此，在这种情况下，可以认为转录的语义准确性更为重要。

(4)https://commonvoice.mozilla.org/

而不是严格遵守正字法标准。有关神经搜索的更多信息，请参见第 6.2 章。

因此，在对自动语音识别系统进行评估时，不仅要考虑单词错误率 (WER)，还要考虑字符错误率 (CER)。表 5.2 和表 5.3 列出了不同模型对各自完整数据集的处理结果。表 5.2 和 5.3 中列出的所有模型都可以通过 "Huggingface "平台[5] 以指定名称获得。Contextnet 和 Citrinet 模型除外，它们从 Nvidia 目录[6]中下载，名称如前。

表 5.2 用域内数据，特别是通用语音数据集训练的语音识别模型的结果。所分析模型的 WER 值在 6.4% 到 15.8% 之间，aces{cer} 值在 2% 到 6% 之间。Fast Conformer [Rek+23]是排名第二的 Conformed Transducer [Gul+20]的优化版本，在 WER 和 CER 方面都取得了最佳结果。排名第三的模型是为这项工作专门训练的 Wav2Vec 模型 [Bae+20]。这三个领先的模型在 WER 方面显示出约 0.5 个百分点的微小差异。

表 5.3：使用 Minds14 数据集的可用 ASR 模型比较。推理时间指的是完整数据集的推理时间。

型号名称	谁	陶瓷	推理时间（秒
openai/whisper-large-v2	0.139	0.088	349
bofenghuang/whisper-medium-cv11-german	0.150	0.081	407
jonatasgrosman/wav2vec2-xls-r-1 b-german	0.154	0.079	78
openai/whisper-medium	0.158	0.100	263
nvidia/stt_en_fastconformer_hybrid_large_pc	0.171	0.082	32
stt_en_conformer_transducer_large	0.181	0.089	47
openai/whisper-small	0.182	0.109	165
stt de contextnet 1024	0.191	0.090	36
jonatasgrosman/wav2vec2-large-xlsr-53-german	0.193	0.095	51
oliverguhr/wav2vec2-large-xlsr-53-german-cv9	0.194	0.094	53
bofenghuang/whisper-small-cv11-german	0.196	0.100	209
STT DE CITRINET 1 024	0.197	0.093	13
facebook/mms-1 b-all	0.225	0.096	64
aware-ai/whisper-small-german	0.246	0.121	189
oliverguhr/wav2vec2-base-german-cv9	0.271	0.123	45

值得注意的是，以前的方法，如 Agarwal 和 Zesch [AZ19]的方法，在通用语音数据集上的 WER 高达 80%，这说明当前模型有了显著的改进。另一个重要方面是模型的推理时间，这是由 16,142 个音频记录组成的整个 Common Voice 13 测试数据集的推理时间。推理时间从最快的模型（Citrinet） 7 分钟到最慢的模型（Whisper）的 98 分钟不等。内部

Shttps://huggingface.co/models
6https://Katalog.ngc.nvidia.com/models

WER 和 CER 都最低的快速 Conformes 只需要约 11 分钟的测试时间。较大的模型，如 MMS 和 Wav2Vec B，每个都有超过 10 亿个参数，大约需要 20 分钟。基于编码器-解码器架构的 Whisper 模型，根据大小不同，感染时间在 47 分钟到 98 分钟之间。

表 5.3 显示了语音识别模型在域外数据（即模型未经过训练的数据集）上的结果。根据这些数据对模型进行的评估表明，大型模型具有明显的优势。Whisper Large V2 模型的词错误率（WER）约为 4%，比较小的 Whisper Medium 提高了 7%，后者专门针对德语进行了训练，词错误率为 5%。在这一类别中，排名第三的最佳模型是带有毫微分参数的 Wave2Vec 变体，该模型针对德语进行了训练，WER 为 5.4%。

对不同语音识别模型的评估表明，自动语音识别取得了重大进展。2023 年发布的快速 Conformed 模型在 TQStÖätQnSÖtZQ 中的字符错误率（CER）较低，同时推理性能较好，但在域外数据上运行时间较长的大型模型的词错误率（WER）较低。

正如 Radford 等人[Rad+22]所描述的，2022 年提出的耳语模型取得了优异的成绩，特别是在域外数据上，这与该架构的开发目标是一致的。这些模型的主要缺点是资源消耗相对较高，执行时间较慢。

Wav2Vec 模型于 20 世纪 9 年代首次推出，是第一个免费提供的自动语音识别（ASR）模型，为德语提供了令人满意的结果。在域内数据上，Wav2Vec 模型继续保持良好的性能，但在域外数据上则新的方法。

5.3.2 优化 CPU 推理模型

在上一节中，我们比较了各种模型的转录准确性和推理速度。由于数据集的大小，推理速度是使用 GPU 确定的。但需要注意的是，通常只有 CPU 可用于 SAR，因此所获得的结果直接移植。有鉴于此，我们开发了一个测试案例，以说明模型作为 VUI 一部分的典型应用场景：在 CPU 上推理短音频记录。在该测试中，一段三秒钟的录音被执行了 50 次，并确定了推理时间的平

均值。模型的执行仅限于四个 CPU 内核，以模拟模型无法整个系统资源的情况。通过这种方法，可以更真实地评估模型在 SARS 情景下的适用性。

从 2021 年开始，在开发 VUI 时Wav2Vec 模型。对于
在基于 CPU 的系统上加速这些模型的执行。